

# Supplementary Material for ForeDroid

## Section A: Detailed Error Analysis

To complement our main paper’s discussion of limitations, we provide a complete breakdown of the false negatives and false positives observed in our evaluation.

**False Negatives (FN): Missed Behaviors.** Out of 639 ground-truth behavior instances, ForeDroid failed to detect 27 ( $\approx 4.2\%$ ). Most of these false negatives fall into categories characterized by runtime-dependent behavior, or incomplete/hard-to-reconstruct call chains. These include: (1) *Clipboard Hijacking and UI Hijacking* (T5, T24): These behaviors often rely on generic API calls (e.g., `ClipboardManager.getPrimaryClip()`, `View.getText()`) that require no special permissions and are excluded from our sensitive API list. As a result, call chains cannot be constructed, leading to under-detection. However, these cases account for only 0.6% (4/639) of all samples. To prevent false positives from over-generalization, we conservatively exclude such APIs. (2) *CPA Fraud and Click Forgery* (T21): CPA (Cost-Per-Action) refers to a billing model in which advertisers pay only when users complete specific actions, such as app installation, registration, or purchase. Malware targeting CPA fraud typically forges these user actions to gain illicit revenue. Such behaviors are commonly implemented via remote payload loading (e.g., `DexClassLoader`) or dynamic script injection, making static reconstruction of behavior paths difficult. Furthermore, GUI-driven click fraud (e.g., fake clicks, overlays) is often unobservable statically. Nevertheless, ForeDroid can still infer such behaviors when call chains include semantically informative methods such as `sendCpaRequest()`. For example, in the ClickAd malware family, our LLM-generated summary correctly inferred: “likely sends forged CPA events to ad servers (e.g., fake installs or activations)”. (3) *Web-Based and Runtime-Loaded Behaviors* (T19, T22): Behaviors such as phishing (T22) and dynamic payload execution (T19) often depend on runtime-loaded web content (e.g., phishing pages or remote JavaScript), which lies beyond the reach of static analysis. Despite this limitation, in some cases, ForeDroid can infer risks when behavior paths contain patterns such as `WebView.loadUrl()` triggered by suspicious entry points (e.g., `SMS_RECEIVED`) or when JavaScript execution is enabled on a remote page. (4) *Location Tracking* (T2) and *Contact Leakage* (T4): In some variants, location inference is achieved indirectly via SIM card info, IP addresses, or WiFi BSSID data, rather than explicit use of `LocationManager`. Since these indirect signals are not included in our sensitive API set, they remain undetected. However, in our dataset, over 95% of location-related behaviors still rely on standard location APIs; the undetected variants account for just 0.4% (3/639).

**Conclusion:** The missed behaviors are primarily attributable to one of three factors: (1) behaviors relying on non-sensitive or generic APIs, (2) runtime-based logic such as remote code loading or dynamic UI rendering, and (3) Web-driven attack surfaces that cannot be fully reconstructed statically.

**False Positives (FP): Over-Inferred Behaviors.** We observe 40 false positives ( $\approx 6.1\%$ ), the majority of which stem from contextual

ambiguity in distinguishing benign versus malicious use cases. Key patterns include: (1) *WebView + JavaScript Injection* (T9) and *Shell Command Execution* (T11): These behaviors are often inferred based on code patterns such as `addJavascriptInterface()` followed by `loadUrl()`, or the presence of `Runtime.exec()`. While these patterns are indicative of malicious behaviors (e.g., remote payload execution or script injection), they are also common in legitimate apps, including developer tools, device management utilities, and web browsers. The absence of additional contextual cues, such as permission abuse or suspicious GUI activity, increases the likelihood of these behaviors being misclassified as malicious. (2) *Tor/Socket Communication* (T10): The presence of networking classes such as `Socket`, `TorSender`, or embedded proxy libraries may indicate covert channels. However, these techniques are equally used by privacy-preserving apps (e.g., VPNs, privacy browsers). Without execution traces or traffic inspection, distinguishing between benign tunneling and malicious exfiltration remains challenging. (3) *Ad-Related Modules* (T8, T21): Advertising SDKs (e.g., Flurry, StartApp) commonly integrate WebView-based rendering and click-tracking mechanisms that resemble ad fraud behaviors. In the absence of precise attribution signals, such as fake click generation or click injection manipulation, benign advertising logic may be falsely flagged as malicious CPA fraud.

**Conclusion:** Most false positives arise from semantically ambiguous behaviors that lack sufficient contextual signals to reliably distinguish between benign and malicious intent. The primary contributing factors include: (1) behavioral overlap between benign and malicious use cases, particularly in advertisement SDKs and privacy-related network modules, and (2) insufficient disambiguating context, such as the absence of GUI intent, permission misuse, or user interaction traces.

## Section B: Example Report (DeepSeek-Generated)

To demonstrate ForeDroid’s interpretability, Table 1 presents an explanation generated by DeepSeek for an app previously classified as malicious. The report includes the functional scenario context, call chains, reasoning via its internal `<think>` phase, and the final anomaly explanation. ForeDroid provides fine-grained behavior reports that support user understanding and expert verification. With comparable performance to ProMal [1]—whose output has shown high user satisfaction and readability in user studies—we expect similar benefits for analysts and downstream users.

## References

- [1] Chunlian Wu, Sen Chen, Jiaming Li, Renchao Chai, Lingling Fan, Xiaofei Xie, and Ruitao Feng. [n. d.]. Beyond Decision: Android Malware Description Generation through Profiling Malicious Behavior Trajectory. *ACM Transactions on Software Engineering and Methodology* ([n. d.]).

**Table 1: Example Report: DeepSeek-Generated Report for Suspicious Behaviors Involving Multiple Call Chains**

| Component               | Details                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Input                   | <p><b>1. Scenario Functional Context:</b><br/> MessageReceiver.onReceive() android.provider.Telephony.SMS_RECEIVED</p> <p><b>2. Sensitive APIs:</b><br/> getLine1Number(), sendTextMessage(), getInstalledApplications(),</p> <p><b>3. Raw Call Chains:</b></p> <ul style="list-style-type: none"> <li>• &lt;com.baseapp.MessageReceiver: void onReceive()&gt; →<br/> &lt;com.baseapp.TorSender: void sendListenedIncomingSMS()&gt; →<br/> &lt;com.baseapp.Utils: java.lang.String getCutIMEI()&gt; →<br/> &lt;android.telephony.SmsManager: void sendTextMessage()&gt;</li> <li>• &lt;com.baseapp.MessageReceiver: void onReceive()&gt; →<br/> &lt;com.baseapp.SmsProcessor: void processCommand()&gt; →<br/> &lt;com.baseapp.SmsProcessor: void processCheckCommand()&gt; →<br/> &lt;com.baseapp.TorSender: void sendCheckData()&gt; →<br/> &lt;com.baseapp.Utils: java.lang.String getPhoneNumber()&gt; →<br/> &lt;android.telephony.TelephonyManager: java.lang.String getLine1Number()&gt;</li> <li>• &lt;com.baseapp.MessageReceiver: void onReceive()&gt; →<br/> &lt;com.baseapp.SmsProcessor: void processCommand()&gt; →<br/> &lt;com.baseapp.SmsProcessor: void processGrabAppsCommand()&gt; →<br/> &lt;com.baseapp.TorSender: void sendInstalledApps()&gt; →<br/> &lt;com.baseapp.Utils: java.lang.String getInstalledAppsList()&gt; →<br/> &lt;android.content.pm.PackageManager: java.util.List getInstalledApplications()&gt;</li> </ul> |
| DeepSeek Reasoning      | <p><b>1. Behavior Logic and Sensitive API Use</b></p> <ul style="list-style-type: none"> <li>• SMS_RECEIVED Trigger: MessageReceiver.onReceive() likely acts as a dispatcher for SMS-based commands (e.g., "GET_IMEI"), parsed via processCommand() to trigger data access and exfiltration.</li> <li>• Sensitive API Coupling: The call chains involve APIs such as getDeviceId(), getLine1Number(), and getInstalledApplications(), all followed by sendTextMessage(), indicating tightly coupled data leakage behavior.</li> </ul> <p><b>2. Control Flow Patterns and Evasion Tactics</b></p> <ul style="list-style-type: none"> <li>• Modular Execution: SmsProcessor.processCommand() dispatches modular commands (e.g., processGrabAppsCommand()), implying command-driven logic, consistent with SMS-based C&amp;C operations.. The use of class name TorSender may suggest obfuscation or misdirection.</li> <li>• Evasive Behavior: All operations occur within onReceive() without persistent storage, minimizing forensic artifacts.</li> </ul> <p><b>3. Match to Known Malware Behaviors</b></p> <ul style="list-style-type: none"> <li>• Banking Trojan Traits: Device info theft and SMS interception may be used to bypass 2FA and hijack accounts.</li> <li>• Spyware Traits: Covert SMS-based data exfiltration and modular command design align with known spyware families (e.g., Cerberus).</li> </ul>                             |
| Output (Anomaly Report) | <p><b>1. Anomaly Summary:</b> This application processes incoming SMS to harvest sensitive user and device data, and sends it through Tor-based channels, suggesting covert monitoring behavior.</p> <p><b>2. Risk Breakdown:</b></p> <ul style="list-style-type: none"> <li>• <b>Privacy Violation:</b> Access to getDeviceId() and getLine1Number() may enable user fingerprinting.</li> <li>• <b>Data Exfiltration:</b> Harvested data is routed through Tor services via TorSender.</li> <li>• <b>SMS Abuse:</b> Interaction with SMS APIs and receivers may support covert command/control operations.</li> </ul> <p><b>3. Mitigation Suggestions:</b></p> <ul style="list-style-type: none"> <li>• Perform permission auditing for SMS, Telephony, and command-related APIs.</li> <li>• Use network monitoring to detect Tor/hidden outbound flows.</li> <li>• Apply runtime tracing to follow command execution and Tor data flow.</li> <li>• Flag apps accessing telephony APIs without user-facing features.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                       |